

Functions that an agent evaluates

A read of `strands-ai-functions` down to its protocols, and `pneuma`: an object-oriented agent layer where a running agent hires its own subagents, on Claude Opus 5 at extra-high thinking effort.

LIBRARY	<code>strands-ai-functions</code> (AWS strands-labs), commit <code>e47dc94</code> , ~22k LOC
PROOF OF CONCEPT	<code>pneuma</code> — <code>~/bonk-fs/projects/pneuma</code>
MODEL	<code>global.anthropic.claude-opus-5</code> on Amazon Bedrock, adaptive thinking, effort <code>xhigh</code>
STACK	<code>uv</code> , Python 3.14, <code>pytest</code> , <code>ruff</code>
RESULT	Oracle satisfied · 38 agent turns · 274,118 output tokens · 39m 28s wall clock

1. What the library actually is

`strands-ai-functions` takes one idea further than most agent frameworks do: a function whose body is empty, whose docstring is the task, and whose return annotation is the contract. The library builds the prompt, runs an agent, and parses a typed result. The interesting part is what it does when the result is wrong.

```
@ai_function
def translate_text(text: str, lang: str) -> str:
    """Translate the text below to the following language: {lang}.
    ---
    {text}
    """

print(translate_text.run_sync("It was the best of times", lang="fr"))
```

Four design decisions distinguish it, and each has a consequence you feel when building on it.

Post-conditions replace hoping

`post_conditions` are ordinary Python callables, or other AI functions, that validate the output. A failure re-prompts the model with the specific errors, up to `max_attempts` times, defaulting to 10 (`src/ai_functions/ai_thread/config.py:115-120`). The feedback is injected as a new user turn tagged `[VALIDATION ERROR]`, so the model revises rather than restarting cold.

This is the feature that makes the library worth using. It also means a broken run reads as a silent hang and an unexplained token bill rather than a traceback — every post-condition failure is a retry, and 10 retries of an xhigh-thinking Opus call is a real amount of money.

Threads are a protocol, not a class

The load-bearing seam is `Spawnable` (`src/ai_functions/protocols.py:299-315`): two synchronous members, `to_thread()` and an `input_shape` property. Anything satisfying it gets hosted as a thread with a handle, a lifecycle, an event log, and token rollup. That is why a Claude Code session (`src/ai_functions/claude_code/claude_code.py:247`) and a Kiro session (`src/ai_functions/kiro/kiro.py:190`) are indistinguishable from a native AI function to the runtime.

The obligation weight sits on the `Thread` that `to_thread()` returns (`src/ai_functions/protocols.py:128-293`): seven members, four async. An implementor owes the runtime cancel checks, pause awaits, result serialization, fork semantics, and teardown.

UNDECLARED PART OF THE CONTRACT

`Spawnable` has no `name` member, yet both spawn paths default the thread's display name from `getattr(target, "name", None)` (`src/ai_functions/runtime/coordinator.py:236`, `src/ai_functions/runtime/worker.py:443`). A `spawnable` without `name` silently loses attribution in event logs and optimizer graphs.

Teams discover each other at runtime

Every thread is handed two tools it did not ask for: `list_threads` to enumerate peers and `send_message` to delegate to one (`src/ai_functions/ai_thread/tools.py:101-249`). Three send modes: `wait` blocks on the reply, `fire_and_forget` discards it, `continue_then_receive` ends the current cycle and resumes when the reply lands.

`wait` carries a waits-for-graph deadlock guard that refuses an edge which would close a cycle. The check-and-register step runs with no intervening `await`, which is what makes it atomic on a single event loop — of two peers waiting on each other, exactly one registers and the other is refused.

Backpropagation over natural language

Named parameters — prompt fragments, learned facts, reusable Python code — live in a pluggable memory backend and are passed to functions as ordinary arguments. After a run, `optimizer.step(result, feedback, backends=[...])` walks the computation graph backward and updates only the parameters responsible (`src/ai_functions/optimizer/textgrad.py:307`). The graph is reconstructed post-hoc from the coordinator's event log, not tracked during execution.

Passing a recalled parameter object preserves the graph edge. Interpolating it into an f-string silently drops it — the value is correct and the gradient never arrives.

2. The gap we found, and filled

The runtime lets an agent *talk* to peers. It does not let an agent *create* one. Grep confirms only `list_threads` and `send_message` are ever injected.

The hook exists and is unused. `ThreadConfig.config_hook` is documented as the place to inject 'spawn_thread' closed over the current runtime and `thread_id`" (`src/ai_functions/ai_thread/config.py:166`), called at the start of every cycle with the live `ThreadContext`. Nothing in the library ships that tool.

`pneuma` ships it. Three tools bound to the cycle context, so the agent doing the hiring is the one the runtime records as parent:

```
hire(role, name, mandate)    # spawn a roster class as a child thread
delegate(name, request)      # run it and read the answer back
dismiss(name)                # terminate it
```

Each `hire` passes `parent_id=ctx.thread_id`. That single argument writes a `ThreadSpawnedEvent` into the hiring agent's log, and that edge is the only thing `subtree_token_usage` walks — so cost attribution up an agent-built tree is a free consequence of hiring correctly rather than bookkeeping we maintain.

WHAT THE LIBRARY WILL NOT DO FOR YOU

No depth limit, no breadth limit, and cancellation does not propagate to children. `coordinator.cancel` sets one thread's signal and cancels its own task; nothing walks `parent_id`. A hiring cap and a `finally` that retires the tree are the caller's job, so `pneuma` enforces both.

3. An object-oriented layer over a decorator-first library

The library's surface is decorator-first: `@ai_function` wraps a module-level function, its docstring is the template, its signature fixes the input shape. An agent is therefore a *function*, and two agents differing only in private data must be two functions.

`pneuma.Agent` inverts it. A subclass declares the static parts in its class body and carries the varying parts as instance attributes; each instance compiles itself into one `AIFunction` whose `prompt_fn` closes over `self`.

```

class Specialist(Agent):
    role = "specialist"
    result_type = Finding
    effort = "xhigh"

    def __init__(self, plane: str) -> None:
        super().__init__(name=f"{plane}-analyst")
        self.plane = plane
        self.evidence = incident.render_plane(plane)    # private to this instance

    def brief(self, request: str) -> str:
        return f"{request}\n\nYour private {self.plane} evidence:\n{self.evidence}"

specialists = [Specialist(p) for p in ("deploys", "metrics", "logs", "traces")]

```

One class, four live threads, four disjoint views of the evidence. Two library details make this work.

Returning a string bypasses the template. `render_prompt` calls `prompt_fn` first; if the return is not `None`, that value is the prompt verbatim, and the docstring path is never consulted. So the prompt can be computed from instance state instead of interpolated from a literal.

The generated signature must be exactly one `str` parameter. `_infer_input_shape` yields `STR_PROMPT` only in that case, and `STR_PROMPT` is the only shape `send_message` can reach. `Agent.build()` therefore synthesizes that signature whatever the subclass does, and asserts the result:

```

fn = AIFunction(prompt_fn, self.result_type, config)
if fn.input_shape is not InputShape.STR_PROMPT:
    raise RuntimeError(f"{self.name} compiled to {fn.input_shape}, not str_prompt")

```

WHY NOT DECORATE A METHOD

`AIFunction` is not a descriptor, so `self` is never bound. A decorated method gets `input_shape == STRUCTURED` (unreachable by peers), and handing it to another agent as a tool fails with missing 1 required positional argument: `'self'`. This breaks the library's own `examples/compose_research_team.py:103-109`. Compiling from a closure in `build()` avoids it.

4. The demo: an incident nobody can solve alone

A war room investigates a fabricated production incident. Four specialists each hold one telemetry plane — deploys, metrics, logs, traces — and cannot read each other's data. The incident lead holds *nothing*: every fact in its verdict has to be obtained from someone else, by interrogating peers or by hiring help.

The dataset is built so that convergence is mandatory rather than encouraged. Each plane alone leaves at least two mechanisms open; only the intersection of three or more narrows it to one. A module-level `self_check()` asserts that the four-way intersection is exactly the planted mechanism *and* that every *pair* of planes still leaves two or more candidates. "You need at least three planes" is machine-checked, not asserted in prose.

PLANE	OPEN	MECHANISMS THAT PLANE ALONE CANNOT RULE OUT
deploys	4	retry amplification, unbounded concurrency, connection pool exhaustion, cache key collision
metrics	3	retry amplification, n plus one fanout, cache key collision
logs	4	retry amplification, n plus one fanout, connection pool exhaustion, cache key collision
traces	4	retry amplification, n plus one fanout, clock skew, connection pool exhaustion

Two decoys are planted to punish single-plane confidence: an `auth-svc` cache-key change with a textbook collision shape that lands 11 minutes *after* symptom onset, and `pricing-svc`, which has the loudest metrics in the fleet and has not been deployed in three days.

Three layers of orchestration in one run

The war room itself is a `Spawnable` with no LLM in its control flow — a plain-Python thread that gets a handle, a lifecycle, and token rollup like any agent. Its `execute` fans out the specialists in parallel with `asyncio.gather`, barriers on the results, then spawns the lead. The fan-out order is ordinary `asyncio`, and therefore reproducible in a way a prompt-driven orchestrator is not.

LAYER	WHO DECIDES	MECHANISM
Deterministic	Our Python	<code>WarRoom.execute</code> — parallel fan-out and barrier
Peer-to-peer	The lead, at runtime	<code>list_threads + send_message</code> , injected by the library
Self-staffing	The lead, at runtime	<code>hire / delegate / dismiss</code> , injected by us

The oracle is a post-condition

The lead's verdict is graded by the library's own retry machinery. The post-condition checks the mechanism against a closed vocabulary, requires citations from at least three planes, requires that decoys were explicitly dismissed, and finally calls `incident.verify`. A failure names the shortfall without leaking the answer, and the model gets another attempt:

```
raise AssertionError(
    "the verdict does not match the evidence: " + "; ".join(problems)
    + ". Re-interrogate the specialists whose planes you leaned on least."
)
```

5. What the run did

Verdict

Culprit checkout-api / chg-4417

Mechanism retry_amplification

Confidence 0.78

Oracle satisfied

Causal chain the lead reconstructed

- 14:12:00 chg-4417 begins a 3-stage canary on checkout-api (d-08 5%, d-09 25% at 14:16, d-10 100% at 14:20). One diff moves five outbound-client keys: http.client.max_retries 2->5, backoff exp_jitter->fixed_50ms, retry_budget_pct 10->disabled, max_inflight 32->unlimited, pool.max_idle 8->16.
- 14:12:03 the rollout-start line (l-05) is the last clean record on any plane; 55s later checkout-api emits the first anomaly anywhere (l-06, 'attempt=3/5'). Both lines are checkout-api, i.e. one host clock, so no clock offset can invert them. Arithmetic on the fixed 50ms backoff bounds attempt 1 of that burst to ~14:12:57.8, worst case 14:12:51.9 - still after 14:12:03.
- Canary instances begin issuing up to 5 attempts per logical pricing.quote call instead of 2, with no retry budget to throttle them. Traces fingerprint this directly: a sick request with items=1 spawns FIVE sequential children, all carrying ONE idempotency key with attempt=1,2,3,4,5 (t-12..t-16) - literal repetition of one logical call, versus the healthy pattern of one child per SKU with distinct keys (t-03/t-04/t-05).
- Outbound volume rises with inbound demand flat: checkout rps_out 1132->1290 while rps_in holds 948->951 (m-checkout-api-03 vs -04), and edge/checkout ingress stays flat in all 11 buckets. This is a change in per-request behaviour, not in traffic.
- The amplification scales with canary DOSE, not with downstream errors. Inverting the mix model gives an implied changed-fraction of 0.035 / 0.331 / 0.849 / 0.963 against the scheduled 0.05 / 0.25 / 1.00 / 1.00 (m-checkout-api-04..-07). At the first deviating bucket the extra calls (+158 rps) exceed pricing's total errors (~3.9/s) by ~40x, so the initial increment is unconditional, not error-triggered.
- Because max_inflight was simultaneously removed (32->unlimited), the retry demand from a small canary fraction was not bounded to that fraction: pricing-svc pool occupancy roughly doubles from 15/48 (l-02, baseline) to 31/48 within ~2 minutes (l-07, 14:14:30), while pricing-svc itself had shipped no change for 46 hours (d-02 to d-15).
- The amplified attempt volume saturates a fixed 48-slot connection pool attested independently by metrics (m-pricing-svc-06 first 48/48 at 14:24, pool_max=48 in all 11 records, never resized) and by pricing-svc's own logs (l-10, l-13, l-17, l-22, l-30, l-36, l-40: 'no connection available in_use=48 max=48', waited_ms rising 812->2411).

8. Connection acquisition, not work, becomes the cost: pool_wait_ms is 94-98% of every sick pricing span (t-12..t-15 380/430/470/500; t-20/t-21 580/590) and residual non-wait work is only 12-28ms, BELOW the healthy 57-61ms end-to-end. Attempts terminate in pool_timeout=true (t-16, t-22). This is a traces-plane measurement, corroborated in kind - not in value - by pricing's pool gauge and PoolTimeout logs.
9. Failed and abandoned attempts feed more attempts, so the multiplier climbs 1.356 -> 2.765 -> 5.231 -> 5.772 and then SATURATES at the configured ceiling: 5.772 is 97.0% of baseline fanout 1.19×5 attempts = 5.95. It is a bounded multiplier pinned to the deployed parameter, and it plateaus while pricing's error rate is still rising (0.243->0.261), so it is not tracking downstream errors.
10. Pricing-svc collapses under the manufactured load: queue full and load shed at depth=512 (l-27), p99 4400ms, error_rate 0.261, saturation 0.98 (m-pricing-svc-08); checkout-api reports 'inflight=214 limit=unlimited' (l-18), 'retry budget disabled; not throttling retries' (l-29) and fanout=5.76 (l-44); pricing confirms duplicate work server-side with 'duplicate request detected ... seen=4/seen=5' (l-20, l-33). edge-gateway is a pure pass-through victim, emitting 502/504s that blame checkout-api (l-19, l-28, l-39) with its own fanout flat at 0.795-0.799.
11. 14:44:08 chg-4423 reverts chg-4417 (d-14, l-46); 14:44:51 the retry keys are restored (l-47 'max_retries=2 backoff=exp_jitter retry_budget_pct=10'). Pricing's pool drains 81 seconds later at UNCHANGED capacity (l-48 'pool recovered in_use=27 max=48'), edge is healthy by 14:48:03 (l-49), and only then is capacity added at 14:52:07 (l-51, d-15 replicas 12->20) - 5m55s AFTER recovery. Demand-side reversal was sufficient; capacity was irrelevant. Full baseline at 14:56 (m-*-11), with the retry ceiling observed back at 1/2 (l-52, t-25).

Decoys dismissed

- chg-4420 (auth-svc, jwt.cache.key sub->sub+aud, ttl_s 60->900, 14:31) - the most seductive decoy, since an audience-blind cache key is exactly the shape of cache_key_collision. Dead on ordering: it POSTDATES the first anomaly by 18 minutes, the first metrics deviation by 15, and the first PoolTimeout by 13. Dead on symptoms: auth-svc shows zero latency, error, load or pool deviation in all 11 buckets (m-auth-svc-01..-11) and logs only healthy lines (l-24, l-35, l-42). Dead on recovery: it was NEVER reverted, yet full baseline was reached at 14:56. Its one real footprint - hit ratio 0.91->0.96->0.97 first moving at 14:44 and never reverting - is an effect of its own 15x TTL extension, not of the incident.
- cache_key_collision as a mechanism - rejected across three planes. No change in the entire 4-day inventory ever altered cart-store's key format; the only key-format change (chg-4420) is the ineligible decoy above. Logs show zero plane-wide matches for 'stale' and no line anywhere shows a wrong or mismatched value served. cart-store first deviates at 14:16, the SAME bucket as everything else and never earlier, never emitted a single ERROR, was never blamed by any service, and recovered to 0.94 with no change of its own. Residual unexplained oddity I could not close: one healthy post-incident span (t-28) reads another trace's key cart:v2:8842 and gets a HIT; spans carry no payload, so correctness is unverifiable.
- cart-store's 2.58x inbound growth with hit ratio falling 0.94->0.70 - the strongest quantitative objection to my verdict, since retried IDENTICAL reads should raise a hit ratio, not lower it. Resolved without a new cause: pricing.quote is a LEAF span with uninstrumented children and pricing-svc has NO rps_out metric on any record, so the pricing->cart-store edge is invisible rather than absent, and cart's growth (2.58x) sits below pricing's intake growth (4.87x) as a partial downstream share would. Falling hit ratio needs no novel keys: requests abandoned mid-flight on pool_timeout never complete their cache write-back, and eviction churn (evictions_5m=41200, l-21) plus widened miss windows do the rest. Honest caveat: that edge is inferred from a coefficient fit, never directly observed.
- chg-4413 (cart-store, cache.max_entries 200k->260k, 09:31) - eligible on timing but dead on dose and recovery: it was at 100% for five hours before onset, so it cannot produce a step change at 14:12, it was never reverted while the fleet fully recovered, and cart-store never led any deviation.

- chg-4415 (edge-gateway, tls.session_cache 8192->16384, 14:02, WITH an unexamined version bump) - the most dangerous predating decoy, landing 10 minutes before chg-4417 with unverified code. Dead on dose-invariance: it was at 100% before onset. Dead on symptoms: edge rps_in (1180-1204) and edge fanout (0.795-0.799) are flat in ALL 11 buckets, so the edge tier neither amplified nor retried; edge shows no distress until 14:20:33 (l-12), 18 minutes after its own deploy, and only ever blames checkout-api. Never reverted, yet recovery completed.
- chg-4408 (checkout-api, feature.flag.split_ship on, 07-27) and n_plus_one_fanout - the flag genuinely multiplies per-request downstream calls, and it was live through the incident. Dead as generator on three planes: traces show baseline children track item count EXACTLY with distinct keys and SKUs (items=3->3, items=2->2, items=1->1), while both sick traces have items=1 yet 5 and 3 children ALL sharing one idem key with attempt=1..N - repetition, not per-item work. Per-tier counts are 1->1->5->0 with multiplication at exactly ONE tier; there is no 1->3->9 cascade. Metrics show demand flat in all 11 buckets, so the item mix cannot have shifted. Decisively, the flag was at 100% for 27 hours before onset and was never reverted while the fleet recovered - it is dose-invariant. It survives only as a possible multiplicand that retries amplified.
- connection_pool_exhaustion - real, directly logged, and the proximate cause of the user-visible errors, but a CONSEQUENCE, not the trigger. Ordering: checkout's egress rose at 14:16 while pricing's pool was still only 19/48; the pool did not pin at 48/48 until 14:24. Capacity: pool_max=48 is constant in all 11 metrics records and in every log line including recovery - nothing shrank it, and no change in the inventory ever altered a pricing-svc pool. Agency: pricing-svc shipped nothing for 46 hours. Recovery: the pool drained at UNCHANGED max=48 within 81s of the retry-key restore and 5m55s before any replica was added. Note the diff RAISED pool.max_idle 8->16, i.e. more idle retention, which relieves rather than causes pressure.
- unbounded_concurrency - the closest rival and the one I cannot fully separate, since max_inflight 32->unlimited shipped in the SAME diff, on the same canary ladder, with the same single rollback. Demoted to permissive co-factor, not generator, on two grounds no other reading explains: (1) removing an in-flight ceiling cannot mint an attempt DENOMINATOR of 5 - logs show the ceiling moving 1/2 -> x/5 -> restored 2 -> 1/2 (l-01, l-06/l-14, l-47, l-52), traces show attempt=1..5 on ONE idem key, and the metrics plateau sits at 97% of exactly 5x baseline; (2) traces show sibling spans strictly SEQUENTIAL with zero overlap, so per-request parallelism is 1. 'inflight=214' (l-18) is not independent support for the concurrency limb - it is rate x inflated latency, a consequence. I record it as the enabling co-factor that let a 5% canary saturate a shared pool.
- clock_skew - measured but non-causal. clock_offset_ms=-118 appears on exactly ONE host (pricing-svc-7) with the IDENTICAL value in a healthy pre-incident span (t-09, 13:52), the sick span (t-12, 14:20) and a healthy post-incident span (t-26, 14:56): zero drift, no correlation with failure, and every other slow span carries no offset at all. A constant offset contributes 0ms to any duration since start and end are read from the same clock, and 118ms is 4.7% of a 2501ms symptom. Logs have zero plane-wide matches for clock/NTP/skew/drift/not-yet-valid/expired, and the decisive ordering pair (l-05 vs l-06) is intra-service on one clock. Where skew IS load-bearing: it corrupts my backoff fingerprint on the one offset host - the first inter-attempt gap is 146ms raw but 28ms skew-corrected, BELOW the 50ms floor, so I rest that fingerprint only on the three clean gaps (51/52/52ms) on non-offset hosts plus 50/50ms in the second sick trace. Two earlier log 'inversions' were withdrawn by the logs plane itself in favour of idempotency-key reuse across request waves, which is intra-service arithmetic no constant offset can produce.
- chg-4421 (edge-gateway, upstream.timeout_ms 1500->2500, 14:34) - postdates the first anomaly by 21 minutes; ineligible. Flagged separately as an effect-precedes-cause anomaly in the evidence itself: a 504 with dur_ms=2501 was already logged at 14:22:47 (l-19), 11m22s BEFORE the 2500ms ceiling was announced (l-38), so the recorded diff cannot explain the ceiling that was already in force. Annotated 'no effect' and never reverted.
- chg-4425 (pricing-svc, replicas 12->20, 14:52) - the intuitive 'we were under-provisioned' story, and doubly ineligible: it postdates onset by 37 minutes AND postdates measured recovery (pool recovered 14:46:12, edge healthy 14:48:03). Capacity was added to an already-healed system.

- chg-4419 (cart-store, trace.sampling 0.10->0.25, 14:26) - postdates cart-store's own first deviation (14:16 bucket) by ~10 minutes, by which time the cache slide was already three buckets deep. Telemetry-only, but it does mean raw span volume is not comparable across 14:26, which I accounted for by reasoning from per-trace SHAPE (children per request, idem keys, attempt counters) rather than span counts.

Evidence cited, by plane

PLANE RECORDS

deploys	d-08/d-09/d-10 (chg-4417 canary 5%/25%/100% at 14:12/14:16/14:20; the five http.client keys incl. max_retries 2->5, backoff->fixed_50ms, retry_budget->disabled, max_inflight->unlimited, pool.max_idle 8->16); d-14 (chg-4423, the ONLY rollback in the record, 14:44); d-02 + d-15 (pricing-svc unchanged for 46h across the incident; no change ever altered a pricing pool size). Decoys: d-04 (chg-4408 split_ship), d-06 (chg-4413 cart cache.max_entries), d-07 (chg-4415 edge tls.session_cache), d-11 (chg-4419 sampling), d-12 (chg-4420 jwt.cache.key), d-13 (chg-4421 timeout), d-15 (chg-4425 replicas) - all never reverted except chg-4417.
metrics	m-*-03 (14:12, all five services baseline - the pre-deploy clean floor); m-checkout-api-04 (first deviation: rps_out 1132->1290 with rps_in flat 948->951); m-checkout-api-03/-04/-05/-06/-07/-08 (fanout ladder 1.194/1.356/2.765/5.231/5.772/5.757, plateau = 97% of 1.19x5); m-checkout-api-01..-11 (rps_in flat 939-957 throughout = demand never moved); m-pricing-svc-04/-05/-06/-07/-08/-09 (pool 19/48 -> 44/48 -> 48/48 first at 14:24, held through 14:44; pool_max=48 constant in all 11; p99 4400, err 0.261, sat 0.98 at -08); m-edge-gateway-01..-11 (rps_in 1180-1204 and fanout 0.795-0.799 dead flat = pure pass-through victim); m-cart-store-03..-08 (rps_in 613->1580, cache 0.94->0.70); m-auth-svc-01..-11 (zero latency/error/load impact ever; hit ratio first moves at 14:44 and never reverts); m-*-10/-11 (near-baseline 14:48, full baseline 14:56).
logs	l-05 (14:12:03 'rollout start version=v2.31.0 canary=5% change=chg-4417' - last clean line anywhere); l-06 (14:12:58 first anomaly, 'attempt=3/5 backoff_ms=50', same host/clock as l-05); l-01 + l-52 (retry ceiling 'attempt=1/2' before and after); l-09/l-14 (attempt=4/5, 5/5); l-15/l-23/l-31 (UpstreamUnavailable after 5 attempts); l-07 (14:14:30 first pricing distress, pool_in_use 31/48 vs baseline 15/48 in l-02); l-10/l-13/l-17/l-22/l-30/l-36/l-40 (PoolTimeout 'in_use=48 max=48', waited_ms 812->2411); l-18 ('outbound inflight=214 limit=unlimited queue_depth=0'); l-27 (pricing load shed, queue depth=512); l-29 ('retry budget disabled; not throttling retries'); l-20/l-33 (pricing 'duplicate request detected ... seen=4 / seen=5'); l-43/l-44 (ratio=4.83, fanout=5.76); l-19/l-28/l-39 (edge 502/504 blaming checkout-api); l-46 -> l-47 -> l-48 -> l-49 -> l-51 (rollback 14:44:08, retry keys restored 14:44:51, pool recovered at unchanged max=48 at 14:46:12, edge healthy 14:48:03, scaling only at 14:52:07); l-34/l-35 (chg-4420 applied 14:31:06, post-onset); l-38 vs l-19 (2500ms ceiling logged at 14:34:09 but already in force at 14:22:47).
traces	t-02..t-07 (healthy tr-7b02: v2.30.6 max_retries=2, items=3 -> three pricing.quote children with DISTINCT idem keys and DISTINCT SKUs); t-09 (items=1 -> 1 child), t-26/t-27 (items=2 -> 2 children) - baseline fanout tracks item count exactly; t-11 + t-12..t-16 (sick tr-9d41: v2.31.0 max_retries=5 backoff=fixed_50ms retry_budget=disabled, items=1 yet FIVE sequential children ALL idem=IDMP-8842-q3 with attempt=1,2,3,4,5; gaps 51/52/52ms on non-offset hosts; pool_wait_ms 380/430/470/500 then pool_timeout=true; residual work 12-28ms vs healthy 57-61ms); t-19 + t-20..t-22 (second sick trace, items=1, three attempts of ONE key, gaps 50/50ms); t-10/t-18 (edge roots merely inheriting child latency +60ms); t-17/t-23 (cart.get exactly 1 per request in sick traces, 14-15ms, never elevated); t-24/t-25 (post-rollback v2.30.6 max_retries=2 healthy); t-09/t-12/t-26 (clock_offset_ms=-118 on pricing-svc-7 only, identical in healthy-before, sick, healthy-after, zero drift); t-28 (unresolved: healthy span reads another trace's cart key with a HIT).

Subagents the lead hired for itself

NAME ROLE MANDATE IT WROTE

timeline	historian	Build a single strict cross-plane timeline for the 2026-07-28 incident from evidence I supply verbatim, and flag any candidate cause whose timestamp postdates the effect it is supposed to explain. You produce ordering verdicts only, never mechanism claims.
----------	-----------	--

NAME	ROLE	MANDATE IT WROTE
intersect	correlator	Intersect findings across the deploys, metrics, logs and traces planes for the 2026-07-28 incident to eliminate every mechanism candidate that survives on one plane but dies on the intersection. Report which candidates are eliminated, by which specific multi-plane conjunction, and which single candidate survives all four planes.
redteam	skeptic	Adversarially review my draft verdict on the 2026-07-28 incident. Attack every unsupported inferential step, name any claim that rests on a single plane, and tell me the specific confidence level the evidence actually licenses.

MEASURE	VALUE
Agent turns	38
Tokens	1,367,046 in / 274,118 out
Subagents hired	3
Delegations	3
Wall clock	39m 28s

```

war-room thinking
thread-87b4a26 spawned child thread-948920b
thread-87b4a26 spawned child thread-5b0790f
thread-87b4a26 spawned child thread-0b687e6
thread-87b4a26 spawned child thread-4699f93
deploys-analyst thinking
metrics-analyst thinking
logs-analyst thinking
traces-analyst thinking
thread-0b687e6 calls Finding(plane=logs, onset=2026-07-28T14:12:58Z, observations=["Baseline is clean
and shows a 2-attempt retry ceiling: ..., suspects=['retry_amplification - strongest fit in my plane: attem)
logs-analyst done
thread-948920b calls Finding(plane=deploys, onset=2026-07-28T14:26:00Z, observations=["d-11 (2026-07-
28T14:26:00Z) is the earliest record in m..., suspects=['retry_amplification -- strongest fit on my plane
al)
deploys-analyst done
thread-5b0790f calls Finding(plane=metrics, onset=2026-07-28T14:16:00Z, observations=['Demand was
constant: edge-gateway rps_in stays 1180-120..., suspects=['n_plus_one_fanout - a per-request increase in
disti)
metrics-analyst done
thread-4699f93 calls Finding(plane=traces, onset=2026-07-28T14:20:47.862Z, observations=['Earliest
failing span on this plane is t-12 (pricing.qu..., suspects=['retry_amplification - strongest fit on this
plan)
traces-analyst done
thread-87b4a26 spawned child thread-1b6ca89
incident-lead thinking
thread-1b6ca89 calls list_threads()
thread-1b6ca89 calls send_message(thread_id=thread-948920ba23ff, message=Incident lead here. I have no
telemetry of my own, so I n...)
thread-1b6ca89 calls send_message(thread_id=thread-5b0790f25268, message=Incident lead here. I have no
telemetry of my own, so I n...)
thread-1b6ca89 calls send_message(thread_id=thread-0b687e6e7a1e, message=Incident lead here. I have no
telemetry of my own, so I n...)
thread-1b6ca89 calls send_message(thread_id=thread-4699f939e02e, message=Incident lead here. I have no
telemetry of my own, so I n...)
deploys-analyst thinking
metrics-analyst thinking
logs-analyst thinking
traces-analyst thinking
thread-948920b calls Finding(cannot_determine=["Whether my inventory is COMPLETE for the fleet.
change..., observations=['COMPLETE INVENTORY: 15 records (d-01..d-15) spanning 20..., onset=2026-07-
28T14:26:00Z)
deploys-analyst done
thread-4699f93 calls Finding(cannot_determine=["Causal order between retry_amplification and
connection..., observations=['[Q1 HEALTHY] tr-7b02 = 7 spans, depth 3: root s-7b02-1 ..., onset=2026-07-
28T14:20:47.)
traces-analyst done
thread-0b687e6 calls Finding(plane=logs, onset=2026-07-28T14:12:58Z, observations=["[1a] EARLIEST
ANOMALOUS LINE - l-06 | 2026-07-28T14:12:..., suspects=['retry_amplification - best-supported in my plane:
ceil)
logs-analyst done
thread-5b0790f calls Finding(cannot_determine="ORDERING WITHIN THE ONSET BUCKET. Four services first
de..., observations=["TIMEZONE: every record's ts ends in 'Z' = UTC/+00:00. B..., onset=2026-07-
28T14:16:00Z)
metrics-analyst done
thread-1b6ca89 calls send_message(message=Follow-up, incident lead. Three precise questions about t...,
thread_id=thread-5b0790f25268)
thread-1b6ca89 calls send_message(message=Follow-up, incident lead. Four precise questions.
Q1 - P..., thread_id=thread-4699f939e02e)
thread-1b6ca89 calls send_message(message=Follow-up, incident lead. Four precise questions about ch...,
thread_id=thread-948920ba23ff)
thread-1b6ca89 calls send_message(message=Follow-up, incident lead. Four precise questions. Your pl...,
thread_id=thread-0b687e6e7a1e)
thread-1b6ca89 calls hire(role=historian, name=timeline, mandate=Build a single strict cross-plane
timeline for the 2026-0...)
thread-1b6ca89 spawned child thread-5a12b64
thread-1b6ca89 calls hire(role=correlator, name=intersect, mandate=Intersect findings across the

```

```
deploys, metrics, logs and ...)
  thread-1b6ca89 spawned child thread-134dfbe
metrics-analyst thinking
... 66 more events
```

Live event stream from the coordinator's log. Every thread appears here, including ones the lead created that no line of our code named.

6. Configuring Opus 5 at extra-high effort

Opus 5 on Bedrock rejects the `thinking` configuration that earlier Claude models accept. Verified against the live API:

```
BedrockModel(  
    model_id="global.anthropic.claude-opus-5",  
    region_name="us-east-1",  
    max_tokens=24_000,  
    streaming=True,  
    additional_request_fields={  
        "thinking": {"type": "adaptive", "display": "summarized"},  
        "output_config": {"effort": "xhigh"},  
    },  
)
```

SETTING	RESULT
<code>reasoning_effort: "xhigh"</code>	Extra inputs are not permitted
<code>thinking.type: "enabled"</code>	Rejected; the error names <code>adaptive</code> + <code>output_config.effort</code> as the replacement
<code>thinking.budget_tokens</code>	Extra inputs are not permitted — <code>max_tokens</code> is the only ceiling
<code>temperature: 0.7</code>	Rejected; adaptive thinking permits temperature 1 only
<code>effort: "extra_high"</code>	Enum is exactly <code>low</code> <code>medium</code> <code>high</code> <code>xhigh</code> <code>max</code>

Effort scales thinking depth measurably. On a fixed prompt at `max_tokens=16000`: `low` produced 114 characters of reasoning in 10.5s, `high` 260 in 19.4s, `xhigh` 662 in 17.4s, `max` 968 in 22.6s. Thinking tokens are drawn from the same budget as the answer, so a tight `max_tokens` truncates the answer rather than the thinking.

INTERACTION WITH STRUCTURED OUTPUT

Strands strips the `thinking` field whenever `tool_choice` forces a tool, citing a Bedrock restriction. That restriction no longer holds for Opus 5 — forced tool use succeeds with adaptive thinking — so the strip is now a silent capability loss on structured-output retries. `output_config` survives.

7. Testing agents without calling an agent

The library ships a scripted-model harness, which is what makes 15 offline tests possible: no Bedrock, no network, 1.3 seconds. `Turn` objects script the model's replies and tool calls; `RuntimeHarness` wires a real coordinator and worker.

```
fn = Echo(name="boss").build().replace(
    model=ScriptedModel([
        Turn(tool_calls=(("hire", {"role": "scripted-helper", "name": "h1", ...})),),
        Turn(tool_calls=(("delegate", {"name": "h1", "request": "when did it start"})),),
        Turn(tool_calls=(("dismiss", {"name": "h1"})),),
        Turn(text="done"),
    ]),
    config_hook=staffing_tools(staff, allow=["scripted-helper"]),
)
```

The suite asserts the properties that would otherwise only fail in production: that two instances of one class produce different prompts, that every compiled agent lands on `STR_PROMPT`, that specialists' evidence is genuinely disjoint, that `hire` writes a parent edge into the hiring agent's log, that the hiring cap holds, and that a bad tool call comes back as recoverable text rather than an exception.

Writing them forced a real design change. Testing `delegate` meant a hired subagent had to run on a scripted model, which meant `Agent` needed a `model()` method rather than hard-coding `opus5()` inside `build()`. The test suite found the missing seam.

8. Defects found in the library

A sixteen-agent documentation pass over the codebase, one agent per perspective, surfaced these. Each is cited in the document that found it.

DEFECT	WHY IT MATTERS
<code>generate_signature_from_model</code> is defined twice; the copy with zero callers sorts required parameters before optional, the reachable copy does not	Any local-code-execution user whose output model declares an optional field before a required one crashes. The two-line fix is already written, in a file nothing imports.
The Claude Code mirror of <code>send_message</code> has no deadlock guard	The Strands path refuses a mutual blocking <code>wait</code> ; two Claude sessions waiting on each other hang with no refusal.
<code>ThreadStatus.TERMINATED</code> and <code>FAILED</code> are declared but never assigned	Termination deregisters the thread instead, so a caller polling <code>handle.is_done()</code> gets <code>ThreadNotFoundError</code> rather than <code>True</code> .
Malformed wire frames are dropped by <code>except Exception: continue</code> with no log line at any level	The library's one completely silent failure path, four lines below a transport-close bug that was fixed for exactly this reason.
<code>@ai_function</code> on a method leaves <code>self</code> unbound	Breaks <code>examples/compose_research_team.py</code> when that method is handed to another agent as a tool.
A <code>--url</code> flag is documented in three docstrings and does not exist; the parameter is accepted and immediately <code>deleted</code>	Documented CLI behavior that silently does nothing.
<code>config.py</code> instructs users to install a <code>[procedural]</code> extra that is not declared	<code>smolagents</code> is already a hard dependency; the instruction is stale.
Returning a t-string from a prompt body, which the tutorial calls "the preferred way", raises <code>TypeError</code>	The documented happy path for Python 3.14 templates does not run.

DEFECT	WHY IT MATTERS
Remote <code>resolve_approval</code> returns <code>True</code> unconditionally after scheduling under <code>suppress(Exception)</code>	Its docstring promises "True iff a matching pending future was resolved".
<code>ThreadContext.metadata</code> is passed by reference from worker state	A thread mutating it mutates worker state for every later cycle.
<code>claude_code/</code> and <code>kiro/</code> — 1,930 LOC across the two external-process boundaries — have zero test references	The two riskiest integration surfaces are the two untested ones.
<code>hatch run check-spec</code> is a required gate in <code>CONTRIBUTING.md</code> and absent from CI	A hand-maintained 10,618-LOC stub tree has no machine gate.

Also worth knowing when building: PyPI `0.3.0` predates `runtime/usage.py`, so the subtree token-rollup helpers exist only on the git HEAD. `pneuma` pins the clone.

9. Assessment

The abstraction earns its keep. Post-conditions turn "the model usually gets this right" into a loop that either satisfies a check or fails loudly, and that alone justifies the dependency for any workflow where a wrong answer propagates. The `Spawnable` protocol is the better idea: making a Claude Code session and a decorated Python function interchangeable to the runtime is a genuine architectural result, and the cost of implementing it is two methods.

The rough edges are alpha-stage rough. Cancellation not propagating to children, no depth or breadth limits on spawning, two declared statuses that are never assigned, and a silent frame-drop path are the kind of thing you discover by reading the source rather than the README — which is what the documentation pass was for. The 22k LOC are unusually well commented, with a repo-root `INVARIANTS.md` naming twelve runtime invariants and their enforcement sites, and that made the defects findable.

Building an object-oriented layer on top was easier than expected, for one reason: `prompt_fn` returning a string bypasses the template machinery entirely. That single escape hatch is what lets an agent's prompt be a method on an object rather than a docstring on a module-level function, and everything `pneuma` does follows from it.

The self-staffing tools are the part we would upstream. The hook is already documented, the parent-edge plumbing already produces correct cost rollup, and roughly 90 lines closes the gap between an agent that can talk to its peers and an agent that can build a team.